

DETERMINISTIC LINTING, AUTHORED IN ENGLISH

RuleSmith

A rulebook a machine *runs* — not one an LLM *reads*

THE TRAP EVERYONE FALLS INTO

"Just put the rules in CLAUDE.md"

Small team, no time to police architecture. The reflex: write the rules into a CLAUDE.md and let the AI handle it.

A CLAUDE.md rule is *prose read by an LLM that pattern-matches text*. It has no control-flow graph. So it cannot answer:

- *Is `close()` on every path, including the early return?*
- *Does the null-check dominate this dereference?*

Same tokens, different flow — bug here, fine there. *You cannot vibe that out of a sentence.*

Same code. Different flow. Opposite verdict.

`examples/java/PaymentGateway.java` — ten methods that each look ordinary line-by-line.

```
$ rulesmith lint --rules $TEN examples/java
warning[resource-leak]: `in` is never closed
= note: close() is on no path (post-dominance)
warning[optional-get-without-ispresent]: `acct.get()` not guarded
...
12 findings

$ rulesmith lint --rules $TEN examples/java-fixed
0 findings
```

The verdict flipped on the `flow`, not the syntax.

WHAT A PROSE RULE STRUCTURALLY CANNOT SEE

Ten flow-sensitive checks

- **resource-leak** — `close()` on every path?
- **optional-get** — `get()` dominated by `isPresent()`?
- **null-deref** — guard misses one path
- **unchecked-downcast** — cast without `instanceof`
- **field-read-before-assign** — read before set
- **non-atomic-shared-update** — read-mod-write
- **guarded-by-lock-held** — field w/o its lock
- **no-string-concat-in-loop** — $O(n^2)$ build
- **no-superfluous-else** — else after return
- **constructor-definite-assignment**

Each needs a **control-flow graph + dominance** — exactly what an LLM reading a sentence does not build.

A real leak, in shipped code

```
$ rulesmith lint --rules resource-leak \  
$BC/.../onedrive/OneDriveConnectorService.java  
  
warning[resource-leak]: `fileInput` (FileInputStream) is never closed  
--> OneDriveConnectorService.java:516  
= note: close() is on no path; resource does not escape  
= help: use try-with-resources
```

The `= note:` is **deterministic evidence** — a graph fact, reproducible every run. Not a vibe.

BEAT 3 — THE CATALOG WRITES ITSELF

Describe a rule in English

```
$ rulesmith add "a switch over an enum must have a default case"
[attempt 1] compiling rule via claude -p...
[ok] violation.java: 1    [ok] clean.java: 0
installed rule 'enum-switch-default' (4 fixtures, all green)
```

The LLM writes the rule — but a **deterministic fixture gate** decides if it ships. That gate is exactly what a CLAUDE.md rule never gets.

WHY YOU CAN TRUST AN AI-WRITTEN RULE

AI builds the checks. Code runs them.

ACTION	NEEDS CLAUDE?	WHY
lint / list / --fix / CI gate	No	pure CFG + AST, deterministic, offline
add "<english>"	Once	codegen, then gated by fixtures
lint --judge	Cached	adjudicate, then cached → AI-free

The everyday workflow is 100% deterministic and runs forever with no API key.

Claude is touched only to *author* a rule and (optionally) to *adjudicate* — both gated, both cached.

Filters its own false positives

```
$ rulesmith lint --rules resource-leak --judge $BC/.../IcebergSourceTask.java
```

```
1 finding filtered as false positive by the judge:
```

```
filtered[resource-leak] IcebergSourceTask.java:101: `reader` never closed
```

```
= judge: not a real issue — OffsetStorageReader is framework-managed;
```

```
Kafka Connect handles the lifecycle
```

Primitives are conservative on purpose; Claude adjudicates the residual, and the verdict is cached so it's reproducible.

BEAT 5 — AND IT FIXES THEM

Safe autofix, the rest suggested

```
$ rulesmith lint --fix --dry-run --rules resource-leak \  
$BC/.../OneDriveConnectorService.java  
  
would fix: 2 resource(s) wrapped in try-with-resources  
2 auto-fixed, 1 needs manual handling (suggest-only)
```

Only the **provably-safe subset** auto-fixes. Everything else stays a suggestion, with rustc-style help + doc link.

Five unmodified production files

5

FILES

236

FINDINGS

13-15

RULES / FILE

0

PARSE ERRORS

Vendored into `examples/real-world/` so the findings reproduce without the repo:

```
rulesmith lint examples/real-world
```

PineconeHelper.java

28 FINDINGS · 14 RULES

```
$ rulesmith lint --rules no-generic-catch,broad-exception-catch examples/real-world/PineconeHelper.java
warning[broad-exception-catch]: Catching broad type 'Exception' traps JVM errors and unrelated bugs.
--> examples/real-world/PineconeHelper.java:70:14
= note: catch (Exception ...)
= help: Catch the narrowest applicable exception type instead.
= see: https://rules.smith.dev/broad-exception-catch

warning[no-generic-catch]: catch block catches generic 'Exception'
--> examples/real-world/PineconeHelper.java:70:14
= note: catch (Exception e) {
= help: Catch a specific exception subclass instead of 'Exception'.
= see: https://rules.smith.dev/no-generic-catch
```

A CLAUDE.md "handle errors properly" rule is unenforceable prose; this flags the **exact site**.

ExcelFileWriter.java

62 FINDINGS · 13 RULES

```
$ rulesmith lint --rules resource-leak examples/real-world/ExcelFileWriter.java
warning[resource-leak]: `customHeader` (StringWriter) is never closed
--> examples/real-world/ExcelFileWriter.java:126:11
= note: no close() call found and the resource does not escape the method
= help: close it in a finally block, or use try-with-resources: try (StringWriter customHeader = ...) { ... }
= see: https://rules.smith.dev/resource-leak
```

Proven via **post-dominance** — `close()` is on no path — not guessed from the name.

TimestampModeProcessor.java

18 FINDINGS · 14 RULES

```
$ rulesmith lint --rules null-deref-needs-dominating-guard examples/real-world/TimestampModeProcessor.java
warning[null-deref-needs-dominating-guard]: 'result' may be null here and is dereferenced without a dominating null guard
--> examples/real-world/TimestampModeProcessor.java:165:30
= note: result.getFrom()
= help: Add a check like `if (result != null)` that dominates this access, or an early `if (result == null) return;`
guard before it
= see: https://rules.smith.dev/null-deref-needs-dominating-guard

warning[null-deref-needs-dominating-guard]: 'result' may be null here and is dereferenced without a dominating null guard
--> examples/real-world/TimestampModeProcessor.java:165:49
= note: result.getTo()
= help: Add a check like `if (result != null)` that dominates this access, or an early `if (result == null) return;`
guard before it
= see: https://rules.smith.dev/null-deref-needs-dominating-guard
```

The canonical *"looks fine line-by-line, NPE on one path"* bug. Only a CFG sees it.

REAL FILE · XML PARSER

XmlUtils.java

48 FINDINGS · 13 RULES

```
$ rulesmith lint --rules unchecked-downcast examples/real-world/XmlUtils.java
warning[unchecked-downcast]: downcast `(Map<String, Object>) el` is not guarded by an instanceof check
--> examples/real-world/XmlUtils.java:119:41
= note: no `el instanceof` test dominates this cast; throws ClassCastException when the runtime type differs
= help: guard with if (el instanceof Map<String, Object>) before casting
= see: https://rules.smith.dev/unchecked-downcast
```

Dominance decides whether the guard actually **covers** the cast.

SQLScriptExecutor.java

80 FINDINGS · 15 RULES

```
$ rulesmith lint --rules broad-exception-catch examples/real-world/SQLScriptExecutor.java
warning[broad-exception-catch]: Catching broad type 'Exception' traps JVM errors and unrelated bugs.
--> examples/real-world/SQLScriptExecutor.java:64:14
= note: catch (Exception ...)
= help: Catch the narrowest applicable exception type instead.
= see: https://rules.smith.dev/broad-exception-catch

warning[broad-exception-catch]: Catching broad type 'Exception' traps JVM errors and unrelated bugs.
--> examples/real-world/SQLScriptExecutor.java:110:14
= note: catch (Exception ...)
= help: Catch the narrowest applicable exception type instead.
= see: https://rules.smith.dev/broad-exception-catch

warning[broad-exception-catch]: Catching broad type 'Throwable' traps JVM errors and unrelated bugs.
--> examples/real-world/SQLScriptExecutor.java:126:14
= note: catch (Throwable ...)
= help: Catch the narrowest applicable exception type instead.
= see: https://rules.smith.dev/broad-exception-catch
```

THE WHOLE THING IN ONE LINE

CLAUDE.md is read.

RuleSmith is run.

113 rules implemented

184-rule catalog mined

0 API keys to run

tree-sitter + CFG/dominance + claude -p · sources: Checker Framework, Effective Java, NASA Power of Ten, OWASP, FP literature